

MTS REPORT ANALYZER

AI-Powered Test Failure Analysis System

Project Specification & Implementation Blueprint

Denis-Marian Chera · April 2026

Software Developer 3 (QA Automation) · Aumovio Automotive

Backend: Python + FastAPI	AI: LLM API (Claude / OpenAI)	Frontend: React	Infra: Docker + docker-compose
----------------------------------	--------------------------------------	------------------------	---------------------------------------

1. Overview & Problem Statement

After every automated test run at Aumovio, a significant number of test cases fail. Investigating each failure manually — opening the HTML report, reading the error message, understanding the root cause, and correlating failures across tests — is time-consuming and requires experience. A junior engineer may spend considerably longer on this than a senior one.

The **MTS Report Analyzer** automates this investigation process. It parses the HTML test reports generated by the MTS Application framework, sends the structured failure data to a Large Language Model (LLM), and presents the analysis in a clean web dashboard — with root cause explanations, failure type classification, cross-test pattern detection, and actionable recommendations.

Without the Tool	With MTS Report Analyzer
Open HTML report, scroll to find FAILs	Dashboard shows all failures instantly on upload
Read each error message manually	LLM explains each failure in plain language
Manually correlate related failures	Automatic pattern detection across tests
Experience required to classify root cause	AI classifies: environment / bug / broken test
No historical context in the report	Severity scoring and recommended actions
Same process repeated per engineer	Shared tool usable by all 20+ engineers

How the LLM Works — No Training Required

Unlike classical ML, the LLM requires **no training data and no historical dataset**. It is a pre-trained model with broad knowledge of software errors, stack traces, and debugging patterns. You provide the failure context (error message, test name, preceding steps) as input, and it reasons about the likely cause — exactly as an experienced engineer would.

INPUT (from report)	LLM OUTPUT
Test: testcase_2429 Error: No mts_measurements/log folder could be found at E:\mts_recording Steps before fail: start_canoe, load_configuration, launch_app	The recording folder was not created before the test ran. This is likely an environment issue — CANoe may not have started the recording session correctly, or the configured path does not exist on this machine. Not a bug in the application. Check CANoe recording settings first.

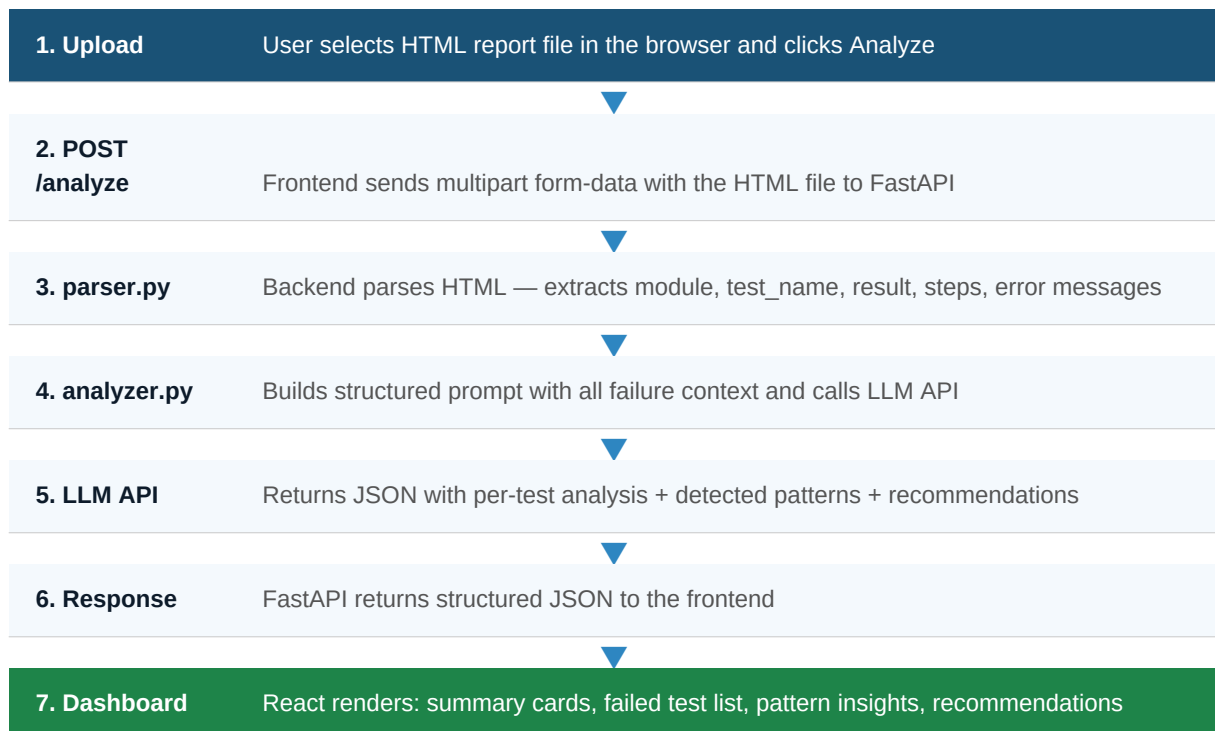
Example: raw error message transformed into an actionable explanation.

2. System Architecture

The system follows a classic three-tier architecture: a React frontend, a FastAPI backend, and an external LLM API. All components are containerized with Docker and orchestrated via docker-compose for single-command startup.

Layer	Technology	Responsibility
Frontend	React + Tailwind CSS	File upload UI, dashboard display, results visualization, pattern cards.
Backend	Python + FastAPI	REST API, HTML parsing, prompt construction, LLM API calls, response structuring.
AI Engine	Claude API / OpenAI API	Natural language reasoning over failure data — root cause, classification, patterns.
Infrastructure	Docker + docker-compose	Containerized deployment; single command startup on any machine.

Request Flow



3. Project Directory Structure

```
mts-report-analyzer/
├── backend/
│   ├── app/
│   │   ├── main.py           # FastAPI entry point + route definitions
│   │   ├── parser.py        # HTML report parser (already implemented!)
│   │   ├── analyzer.py      # LLM integration + prompt engineering
│   │   └── models.py         # Pydantic request/response models
│   ├── Dockerfile           # Backend container definition
│   └── requirements.txt
├── frontend/
│   ├── src/
│   │   ├── App.jsx          # Root component + routing
│   │   └── components/
│   │       ├── UploadPanel.jsx # Drag-and-drop file upload
│   │       ├── SummaryCard.jsx # Run overview (total/passed/failed)
│   │       ├── TestResultList.jsx # Per-test failure analysis
│   │       └── PatternInsights.jsx # Cross-test pattern cards
│   ├── Dockerfile          # Frontend container definition
│   └── package.json
├── docker-compose.yml      # Orchestrates backend + frontend
└── README.md
```

Key Reuse: parser.py

The HTML parser developed during the **Test Suite Optimizer** project is directly reused here. It already correctly identifies the final verdict row per test case, extracts the module name, test name, result, and step count from real MTS Application reports — verified on production data. This is a concrete example of building reusable components across projects.

4. Backend — FastAPI

REST API Endpoints

Method	Endpoint	Description	Response
POST	/analyze	Accepts HTML report file upload; runs full parse + LLM analysis pipeline.	AnalysisResult JSON
GET	/health	Health check endpoint — used by Docker to verify backend is running.	{ "status": "ok" }

Pydantic Response Model

The backend returns a strongly-typed JSON structure to the frontend:

```
class TestFailureAnalysis(BaseModel):
    test_name: str
    module: str
    result: str # 'PASS' or 'FAIL'
    root_cause: str # LLM explanation in plain language
    failure_type: str # 'environment' | 'bug' | 'broken_test'
    severity: str # 'critical' | 'medium' | 'minor'
    recommendation: str # What to check / fix first

class PatternInsight(BaseModel):
    pattern_title: str # e.g. 'CANoe connection failures'
    affected_tests: list[str] # Test names sharing this pattern
    explanation: str # Why these tests are grouped

class AnalysisResult(BaseModel):
    total_tests: int
    passed: int
    failed: int
    run_summary: str # LLM-generated overall summary
    test_analyses: list[TestFailureAnalysis]
    patterns: list[PatternInsight]
    top_priority: str # What to fix first
```

analyzer.py — Prompt Engineering

The quality of the LLM output depends entirely on the quality of the prompt. The analyzer module builds a structured, context-rich prompt for every analysis request:

```
You are an expert QA engineer analyzing test results for the MTS Application,
an automotive testing tool. Analyze the following test run results.
```

```
APPLICATION CONTEXT:
```

- MTS Application: automotive signal analysis and recording tool
- Test environment: Windows desktop, CANoe integration, Ethernet/CAN bus
- Test framework: Python automation with pywinauto

FAILED TESTS:

{for each failed test}

Test: {test_name} | Module: {module} | Steps: {step_count}

Last step before failure: {last_step}

Error description: {error_message}

Return a JSON object with: per-test root_cause, failure_type, severity, recommendation, plus cross-test patterns and top_priority.

5. Frontend — React Dashboard

Components

Component	Description
UploadPanel	Drag-and-drop zone for HTML report files. Shows upload progress. Triggers analysis on file selection.
SummaryCard	Top-level run overview: total tests, passed (green), failed (red), run date extracted from filename.
TestResultList	Expandable list of all failed tests. Each card shows: test name, module, failure type badge, root cause explanation, severity indicator, and recommendation.
PatternInsights	Cards grouping tests that share the same root cause. Highlights systemic issues vs. isolated failures.

Dashboard Layout

The dashboard is organized in four visual zones, top to bottom:

Zone 1 — Upload	Centered drag-and-drop panel. Visible only before a report is loaded.
Zone 2 — Run Summary	Three stat cards: Total Tests / Passed / Failed. Plus the AI-generated overall summary paragraph.
Zone 3 — Pattern Insights	Horizontal scrollable row of pattern cards. Each card lists affected tests and the shared root cause.
Zone 4 — Failed Tests Detail	Vertical list of expandable failure cards, sorted by severity (critical first). Each card shows full analysis.

Failure Type Badge Colors

Failure Type	Color	Meaning
environment	AMBER	Problem with test setup, CANoe, folders, paths — not a bug in the app.
bug	RED	Genuine defect in the MTS Application that needs to be reported.

Failure Type	Color	Meaning
broken_test	PURPLE	The test script itself is outdated or incorrect — needs maintenance.

6. LLM Integration

How the LLM Reasons Without Training Data

A common misconception is that AI tools require training data specific to your application. The LLM used here is a **pre-trained foundation model** with broad knowledge of: software engineering, debugging patterns, automotive tooling (CANoe, ARXML, CAN bus), Python stack traces, and test automation. It does not need to have seen MTS Application specifically — it reasons from first principles, exactly as an experienced engineer would when reading an unfamiliar error message.

What Makes the Analysis Accurate

- **Structured context:** The prompt includes application domain, tool names (CANoe, MTS, pywinauto), and the sequence of steps before the failure — not just the error message.
- **Error message quality:** MTS Application produces descriptive English error messages ('No mts_measurements/log folder could be found at...') which are immediately interpretable by the LLM.
- **Pattern detection:** By sending all failures in a single prompt, the LLM can correlate tests that failed for the same reason — something impossible when analyzing one test at a time.
- **Structured output:** The LLM is instructed to return JSON, which prevents free-form hallucination and ensures consistent, parseable responses.

LLM API Options

Provider	Model	Strengths	Notes
Anthropic	Claude Sonnet	Excellent at structured JSON output, strong reasoning, detailed explanations	Recommended
OpenAI	GPT-4o	Industry standard, wide documentation, very capable	Good alternative

Example LLM JSON Response

```
{
  "run_summary": "2 of 3 tests failed. Both failures appear to be environment issues related to missing recording folders and CANoe configuration, not bugs in the MTS Application itself.",
  "test_analyses": [
    {
      "test_name": "testcase_2429",
      "root_cause": "Recording folder not found at configured path.",
      "failure_type": "environment",
      "severity": "critical",
      "recommendation": "Verify CANoe recording session started before test."
    },
  ],
}
```

```
{
  "test_name": "testcase_2726_signals_info_from_customer_arxml_online",
  "root_cause": "Expected CSV file not generated – data export step failed.",
  "failure_type": "environment",
  "severity": "medium",
  "recommendation": "Check CSV export path permissions and disk space."
},
"patterns": [
  {
    "pattern_title": "Recording/export path issues",
    "affected_tests": ["testcase_2429", "testcase_2726_signals_info..."],
    "explanation": "Both tests require file system write access that was unavailable."
  }
],
"top_priority": "Investigate file system permissions and CANoe recording setup."
}
```

7. Docker & Containerization

Both the backend and frontend are containerized with Docker and orchestrated via docker-compose. This means the entire application — with all dependencies — can be started with a single command on any machine, regardless of the local Python or Node.js setup.

docker-compose.yml

```
version: '3.8'
services:
  backend:
    build: ./backend
    ports:
      - '8000:8000'
    environment:
      - LLM_API_KEY=${LLM_API_KEY}
    volumes:
      - ./backend:/app

  frontend:
    build: ./frontend
    ports:
      - '3000:3000'
    depends_on:
      - backend
    environment:
      - REACT_APP_API_URL=http://localhost:8000
```

Starting the Application

```
# Clone the repository
git clone https://github.com/denis-chera/mts-report-analyzer

# Set your LLM API key
export LLM_API_KEY=your_api_key_here

# Start everything with one command
docker-compose up

# Application available at:
#   Frontend:  http://localhost:3000
#   Backend:   http://localhost:8000
#   API docs:  http://localhost:8000/docs (FastAPI auto-generated)
```

Without Docker	With Docker
Install Python 3.11 + pip + venv	Just: docker-compose up
Install Node.js + npm	Works identically on Windows, Mac, Linux

Without Docker	With Docker
pip install + npm install	No dependency conflicts
May break on different OS versions	Reproducible environment always
Hard to share with colleagues	Anyone can run it in 2 minutes

8. Implementation Plan & Timeline

Phase	Tasks	Est. Effort	Status
Phase 1 Backend Core	Set up FastAPI project structure. Integrate parser.py. Implement /analyze endpoint. Add Pydantic models.	3–4 hrs	READY TO START
Phase 2 LLM Integration	Implement analyzer.py. Design prompt template. Handle LLM API calls. Parse and validate JSON response.	4–5 hrs	PENDING
Phase 3 React Frontend	Create React app. Build UploadPanel, SummaryCard, TestResultList, PatternInsights components.	5–6 hrs	PENDING
Phase 4 Docker	Write Dockerfiles for backend and frontend. Create docker-compose.yml. Test full stack locally.	2–3 hrs	PENDING
Phase 5 Polish	Error handling, loading states, edge cases (empty report, all pass). README with setup instructions.	2–3 hrs	PENDING

Total estimated effort: **16–21 hours** of focused development. Phase 1 has a head start because parser.py is already implemented and tested on real MTS data.

9. Technology Stack

Technology	Version	Role	Why chosen
Python	3.11+	Backend language	Primary language at Aumovio; rich ecosystem for parsing and API development.
FastAPI	0.110+	REST API framework	Modern, async, auto-generates /docs UI, Pydantic integration, industry standard for Python APIs.
BeautifulSoup4	4.x	HTML parsing	Reliable DOM parsing; already used in parser.py.
Pydantic	2.x	Data validation	Enforces strict typing on LLM JSON output; prevents malformed responses from reaching frontend.
Claude / GPT-4o	latest	AI analysis engine	Pre-trained reasoning capability; no fine-tuning required; strong at structured JSON output.
React	18+	Frontend UI	Industry-standard component model; large ecosystem; familiar to most frontend developers.
Tailwind CSS	3.x	Styling	Utility-first; fast to build clean dashboards without custom CSS files.
Docker	24+	Containerization	Reproducible environments; single-command startup; required skill in most SDET job descriptions.
docker-compose	2.x	Orchestration	Manages multi-container setup (backend + frontend) with one config file.

10. Portfolio & CV Value

This project demonstrates a full-stack engineering skillset anchored in a real, domain-specific problem — making it significantly more compelling than generic tutorial projects. Every component maps to a concrete, marketable skill.

Component	Skill Demonstrated	Interview Talking Point
HTML parser reuse	Software design, reusability, pragmatic engineering	'I reused the parser from a previous project — it was already tested on production data.'
FastAPI backend	REST API design, async Python, OpenAPI, Pydantic	'The API auto-generates documentation at /docs — any engineer can explore it instantly.'
Prompt engineering	LLM integration, AI product thinking, context design	'The quality of the analysis depends on context — I structured the prompt to include domain knowledge the LLM needs.'
Pydantic models	Type safety, data validation, defensive programming	'LLM output is validated against a strict schema before it ever reaches the frontend.'
React dashboard	Frontend development, component architecture, UX thinking	'I designed the layout around the engineer's workflow — upload, scan summary, investigate failures.'
Docker + compose	Containerization, DevOps, reproducible environments	'Anyone can run the full stack with docker-compose up — no local setup required.'
Domain context	Automotive QA expertise, real-world problem solving	'This solves a problem I face every day at work — it's not a toy project.'

One-Line CV Description

Built an AI-powered test failure analysis tool that parses MTS Application HTML reports, uses an LLM API to explain root causes and detect cross-test failure patterns, and presents results in a React dashboard — fully containerized with Docker.

